DBMS MINI-PROJECT REPORT

STUDENT ACADEMIC RECORD MANAGEMENT SYSTEM

1. Overview

The Student Academic Record Management System is a database-driven application designed to organize student information in one place. It keeps academic history, semester-wise performance, subject marks, study gaps, seminar participation, and extracurricular activities in structured records. A web interface allows an administrator to find and maintain information without handling separate files. The system uses SQLite for persistent storage and can export selected records to Google Sheets when the required API settings are available.

2. Project Goals

Create a single, organized repository for student information.
Record previous education details, including SSC, HSC, and diploma information where applicable.
Maintain semester results and individual subject marks.
Document academic interruptions or drop years.
Keep a history of seminars and co-curricular or extracurricular participation.
Provide search facilities using a student name or roll number.
Present academic performance through tables and graphs where implemented.
Allow records to be exported to Google Sheets after integration is configured.

3. Requirements

Hardware: A desktop or laptop with at least 4 GB RAM. An internet connection is needed for online spreadsheet export.

Software: Windows, Linux, or macOS; Node.js 18 or later; npm; a modern web browser; SQLite; and Google Sheets API credentials for the optional export feature.

4. Application Structure

The application follows a web-based client/server arrangement. HTML, CSS, and JavaScript form the user interface. The server side runs on Node.js with Express and processes requests from the browser. SQLite stores the application records, while the Google Sheets API is an optional external service used to export data.

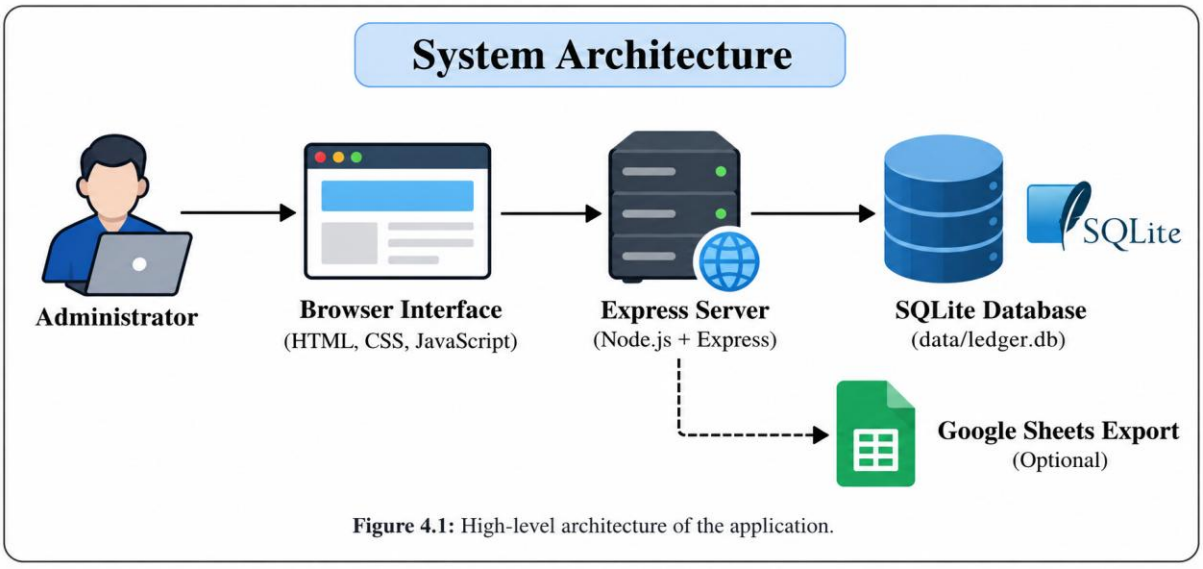


Figure 4.1: High-level architecture of the application.

5. Database Design

SQLite is used as the primary data store. The database is organized around student details and related records for education history, semester results, subject marks, academic gaps, seminars, and activities. This arrangement separates different kinds of information and allows related records to be retrieved together.

5.1 Keys and Relationships

A primary key identifies each row uniquely. Foreign keys associate dependent records with the relevant student or semester record, helping preserve relationships between tables.

5.2 Integrity Rules

Depending on the table definition, constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, ON DELETE CASCADE, and ON DELETE SET NULL can be used to prevent invalid values and define how related records behave when data is deleted.

5.3 Normalization

Normalization helps limit duplicate data and update anomalies. First Normal Form (1NF) requires atomic values; Second Normal Form (2NF) removes partial dependencies on a composite key; and Third Normal Form (3NF) removes transitive dependencies. The final normal-form status must be confirmed against the actual SQL schema.

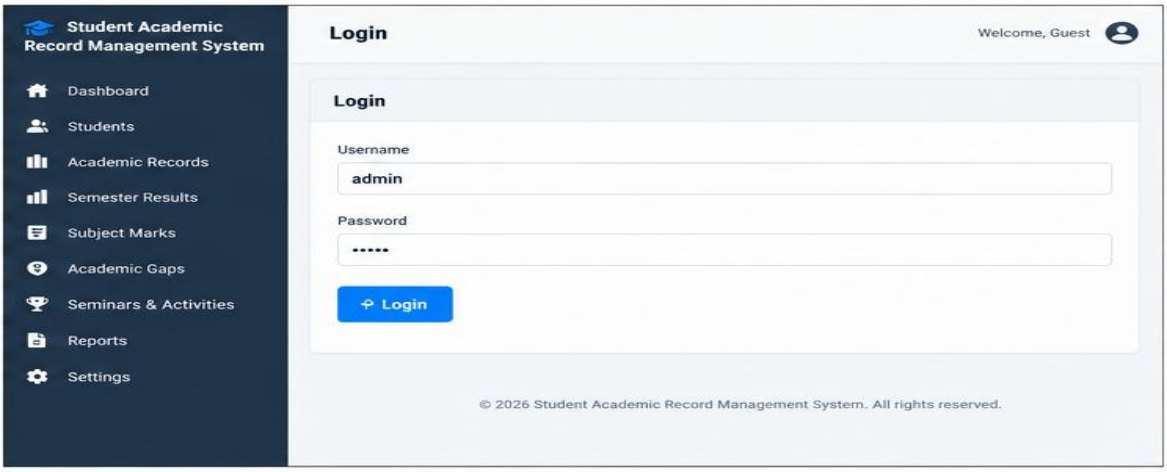
5.4 Storage Implementation

The Node.js application accesses SQLite through the better-sqlite3 library. The database structure is defined in schema.sql, and the default database location is data/ledger.db. The supplied schema and configuration should be checked before making claims about individual constraints or tables.

6. Main Interface Screens

6.1 Login Screen

The login screen provides the entry point for authorized local use.



6.1 Login Screen

6.2 Administration Dashboard

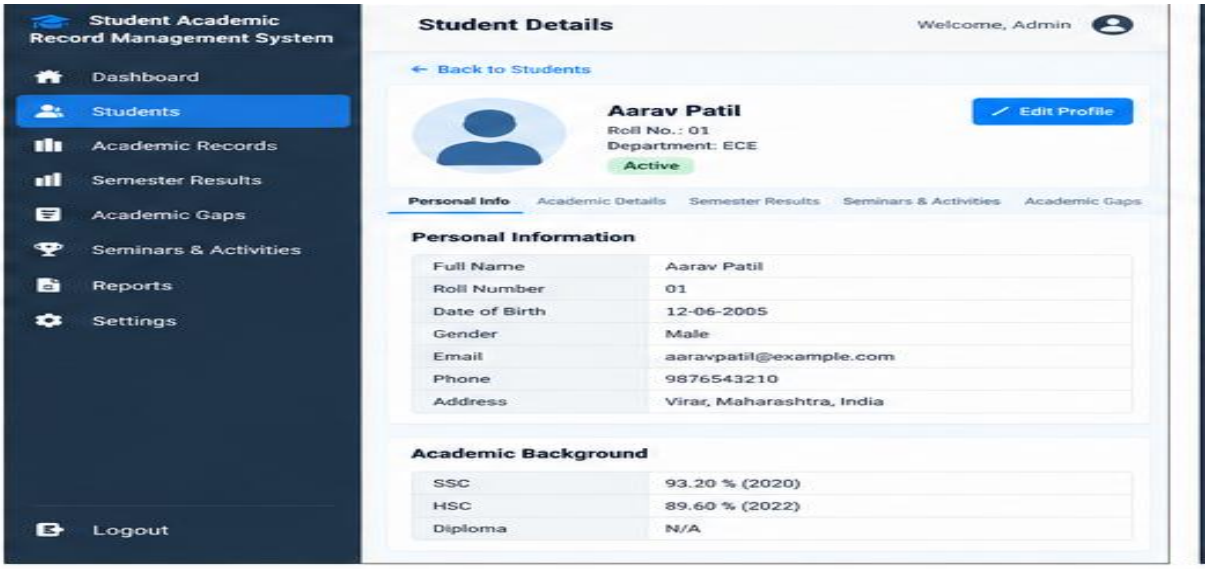
The dashboard gives access to student records and available management options.



6.2 Administration Dashboard

6.3 Student Information View

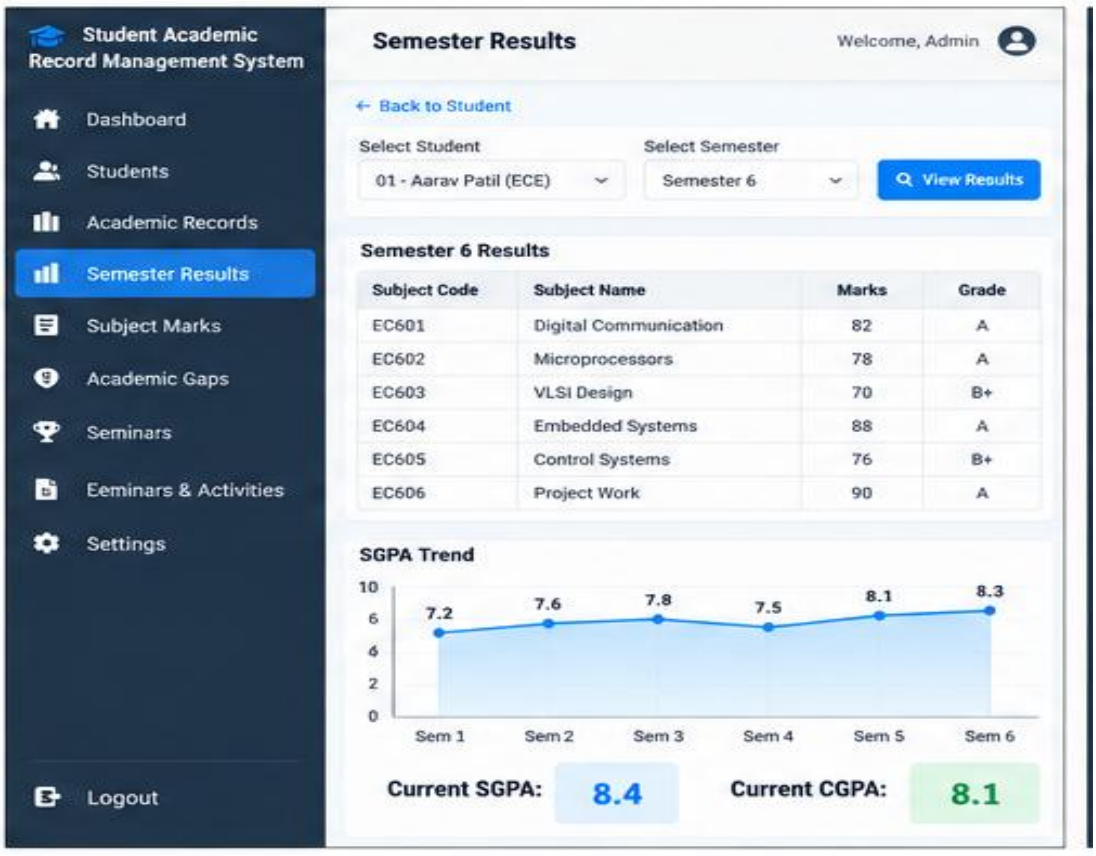
This view presents a student’s profile and educational background. Add a screenshot.



6.3 Student Information View

6.4 Semester Performance

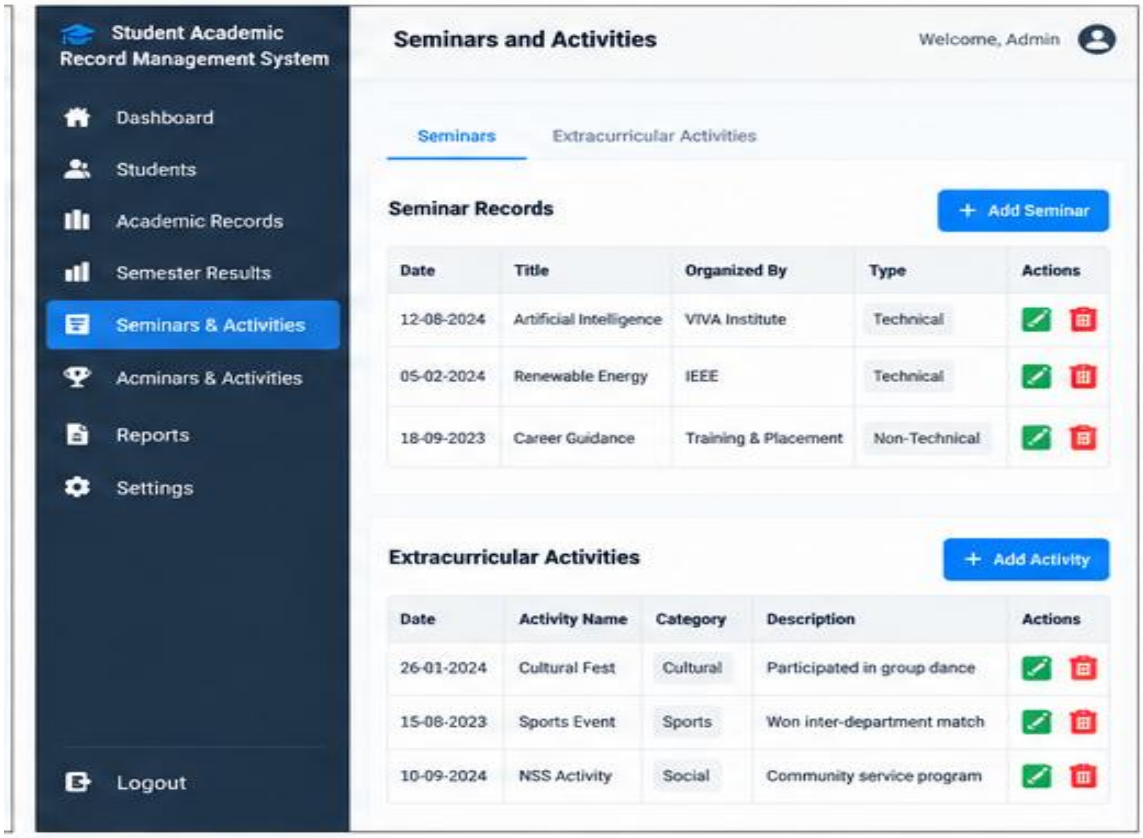
Semester-wise results and subject marks can be reviewed here; include a performance graph if it is available in the implementation.



6.4 Semester Performance

6.5 Seminars and Activities

This section records participation in seminars and other activities.



6.5 Seminars and Activities

7. Testing and Observations

Testing should be performed on the running application to check that common actions behave as intended. Record the actual outcome of each test rather than assuming that every feature has passed.

Test	Expected behaviour	Observed result
Search by name or roll number	Matching student records are shown.	[Fill after testing]
Add or update a record	Valid changes are saved to the database.	[Fill after testing]
Delete a record	The configured deletion rule is applied.	[Fill after testing]
Enter invalid marks/SGPA	Invalid values are rejected if validation is implemented.	[Fill after testing]
Export to Google Sheets	Records are exported when API configuration is correct.	[Fill after testing]

Result summary: Run the project test command (npm test, if configured) and enter the real test output here. Do not report tests as successful unless they have been executed.

8. Basic User Procedure

Install Node.js version 18 or later.
Open the project directory in a terminal.
Install dependencies using npm install.
Start the server using npm start.
Open http://localhost:3000 in a browser.
Use the locally configured login and available screens to search or manage records.
Use the spreadsheet export option only after Google Sheets credentials and the spreadsheet ID have been configured.
Use only the demonstration credentials documented in the project README for local testing.
Replace demonstration authentication with a secure configuration before any public deployment.

9. Google Sheets Export

The project can export data to Google Sheets through the Google Sheets API after configuration. Setup generally requires service-account credentials and the target spreadsheet ID. SQLite remains the primary database. Direct changes made in SQL do not automatically initiate an export, and edits made in the spreadsheet are not imported back into SQLite in the current one-way arrangement.

Figure 9.1: Add a screenshot of a successful spreadsheet export, if available.

10. Constraints and Possible Improvements

Current Constraints

Spreadsheet export depends on correct API configuration.
Database changes do not automatically synchronize with Google Sheets.
Spreadsheet-side edits are not synchronized back to the database.
SGPA is entered manually rather than calculated automatically.
Demonstration login credentials are not appropriate for a public system.

Future Enhancements

Introduce scheduled or event-based export synchronization.
Add carefully validated two-way synchronization if required.
Calculate semester results using the institution’s credit and grading rules.
Strengthen authentication, roles, and access control.
Add downloadable reports, filters, and more detailed visual analytics.

11. Conclusion

The Student Academic Record Management System illustrates how a relational database can support the organized handling of academic and activity information. Its design provides a practical example of tables, keys, relationships, integrity constraints, normalization, and

optional integration with an external spreadsheet service. The final report should be updated with screenshots, schema details, and test results from the actual running project.

12. References

SQLite Documentation — https://www.sqlite.org/docs.html
Node.js Documentation — https://nodejs.org/docs/
Express.js — https://expressjs.com/
Google Sheets API — https://developers.google.com/workspace/sheets/api
MDN Web Docs: HTML — https://developer.mozilla.org/en-US/docs/Web/HTML
MDN Web Docs: CSS — https://developer.mozilla.org/en-US/docs/Web/CSS
MDN Web Docs: JavaScript — https://developer.mozilla.org/en-US/docs/Web/JavaScript